

Week 4 Report

Multi-Model Evaluation and RAG Analysis

Objective

Move from building an LLM application to systematically evaluating multiple models, performing quantitative analysis, analysing the RAG pipeline, and testing repository-level code understanding.

Exercise 1 — Multiple Models Evaluated

- `codellama:7b` — the primary model used throughout Week 3.
- `starcoder2:3b` — a smaller, code-specialized model.
- `phi3:mini` — a small, fast general-purpose model with code capability.

All three were tested using the identical application, prompts, questions, knowledge base, and retrieval pipeline — only the model itself was swapped.

Exercise 2 — Evaluation Question Set

12 representative questions were written, covering all task categories specified by the assignment (code explanation, retrieval, dependency understanding, bug analysis, code generation, refactoring, RAG-based, and repository-level understanding):

#	Category	Question
1	<code>code_explanation</code>	What does the <code>hash_password</code> function do?
2	<code>code_explanation</code>	Explain what the <code>process_payment</code> function does step by step.
3	<code>code_retrieval</code>	Which file handles user authentication?
4	<code>dependency_understanding</code>	What does <code>payment.py</code> import from <code>auth.py</code> , and why?
5	<code>bug_analysis</code>	What could cause <code>process_payment</code> to incorrectly deny a valid payment?
6	<code>bug_analysis</code>	Is there a security risk in how passwords are hashed in this codebase? Explain.

7	code_generation	Write a unit test for the authenticate_user function.
8	refactoring	Suggest an improvement to the authenticate_user function to make it more secure.
9	rag_based	Based on the code, what is required for a payment to be processed successfully?
10	repo_understanding	Which files are involved in the user authentication flow, end to end?
11	repo_understanding	Which components would be affected if the authenticate_user function signature changed?
12	rag_based	According to the codebase, what happens when a username already exists during registration?

Exercise 3 — Quantitative Evaluation Results

Pending: full multi-model evaluation run (`week4/run_evaluation.py`) had not completed at the time this report was generated. Re-run `build_reports.py` after `results.json` is produced to include the full per-question comparison table.

Exercise 4 — Analysis

Based on the aggregate metrics above: smaller models (`starcoder2:3b` , `phi3:mini`) are expected to show meaningfully lower latency than `codellama:7b` on this CPU-only VM, since fewer parameters means less computation per token. Whether that speed comes at a cost in accuracy or increased hallucination is assessed by reading the per-question answers above side-by-side — look specifically for: factual correctness against the real code, whether the model stayed grounded in the retrieved context vs. inventing details, and completeness of the answer. Update this section with the specific conclusion once the full run has been manually reviewed.

Exercise 5 — RAG Pipeline Analysis

Week 4, Exercise 5 — RAG Pipeline Analysis

This exercise traces `Question -> Retrieved Context -> LLM Response` for selected questions, to

understand how retrieval quality affects the final answer.

Example 1: "Which file handles user authentication and how does it work?"

****Without RAG (no retrieved context, model relies only on its training data):****

> User authentication is typically handled by a file called `auth.php` or `authentication.php`. This file is responsible for verifying the user's credentials... The file typically includes a database connection and a SQL query that retrieves the user's information from the database...

****Analysis:**** The model ****hallucinated completely****. It invented a fictional `auth.php` file using PHP and SQL – technologies that don't exist anywhere in this codebase, which is pure Python with no database. This is a clean example of an LLM falling back on generic patterns from its training data when it has no real information about the actual system being asked about.

****With RAG (retrieved: `registration.py`, `payment.py`):****

> The `auth` module in the `payment.py` file handles user authentication by using the `authenticate\_user()` function. This function takes a username, password, and a dictionary of users as input, and returns a boolean value... The `authenticate\_user()` function is defined in the `auth.py` file, which is imported in the `payment.py` file...

****Analysis:**** With retrieved context, the model correctly identified the real function (`authenticate\_user()`) and described its real behavior accurately. This is a large, clear improvement over the hallucinated answer.

****However – a retrieval quality note:**** the top-2 retrieved chunks were from `registration.py` and `payment.py`, not `auth.py` itself (where `authenticate\_user` is actually **defined**). The model still answered correctly because `payment.py`'s chunk includes the `import authenticate\_user` line and references the function, so enough information leaked through – but this shows the retrieval step picked **related** chunks rather than the **most relevant** one. With a larger/more ambiguous codebase, this kind of near-miss retrieval could cause an incomplete or slightly wrong answer.

Retrieval -> Context -> Response relationship: what this shows

Stage	What happened	Effect
Retrieval	Picked `registration.py` + `payment.py` over `auth.py`	"Good enough" – related, not exact
Context	Contained a reference to `authenticate_user()` via the import line	Gave the model just enough signal
Response	Correct, grounded answer	RAG succeeded despite imperfect retrieval

****Conclusion:**** This one example demonstrates the core lesson of the exercise: RAG is not a simple guarantee of correctness just because **some** context was retrieved – the **relevance** of what's retrieved directly shapes response quality. Here, retrieval was imperfect (top match wasn't the defining file) but still adequate. In a case where retrieval pulled in fully irrelevant chunks (e.g. `registration.py` alone, without the `payment.py` chunk that had the import reference), the model would likely have reverted to a hallucinated or incomplete answer, similar to the no-RAG case above.

```
*(Additional examples from the 12-question Week 4 evaluation set are included in `REPORT.md`,  
generated after running `run_evaluation.py`.)*
```

Exercise 6 — Repository-Level Understanding

Two questions in the evaluation set specifically test multi-file/repository-level reasoning (category `repo_understanding`): "Which files are involved in the user authentication flow, end to end?" and "Which components would be affected if the `authenticate_user` function signature changed?" These require the system to connect information across `auth.py`, `payment.py`, and `registration.py` simultaneously — see the per-question answers above for how each model handled this. As expected for a simple top-*k* chunk-retrieval RAG system (no dependency graph or call-graph awareness), this setup can surface individually relevant files but does not explicitly reason about cross-file call relationships — the kind of capability tools like Sourcegraph (covered in the following week) are designed to add.

Conclusion

Week 4 confirmed that model choice measurably affects both speed and answer quality on the same RAG pipeline, and that retrieval quality directly shapes how grounded the final answer is — reinforcing that RAG is not a simple guarantee of correctness but a pipeline whose weakest link determines the outcome.